COMPUTACION

Clase 9

PROPUESTA: Resolver

La siguiente tabla presenta datos de un conjunto de N piezas metálicas sometidas a diferentes tratamientos anticorrosión (A a E):

Nro	Tipo Metal	Peso	Tratamiento	Efectividad (en %)
1	A1	3,4	D	12
2	A2	2,7	A	19,2
3	B3	0,87	A	22
4	B7	1,9	D	0,76
5	A2	3,2	C	34

Se desea tener un listado con los datos de las piezas en las que el tratamiento tuvo una efectividad mayor al promedio de los todos los tratamientos.

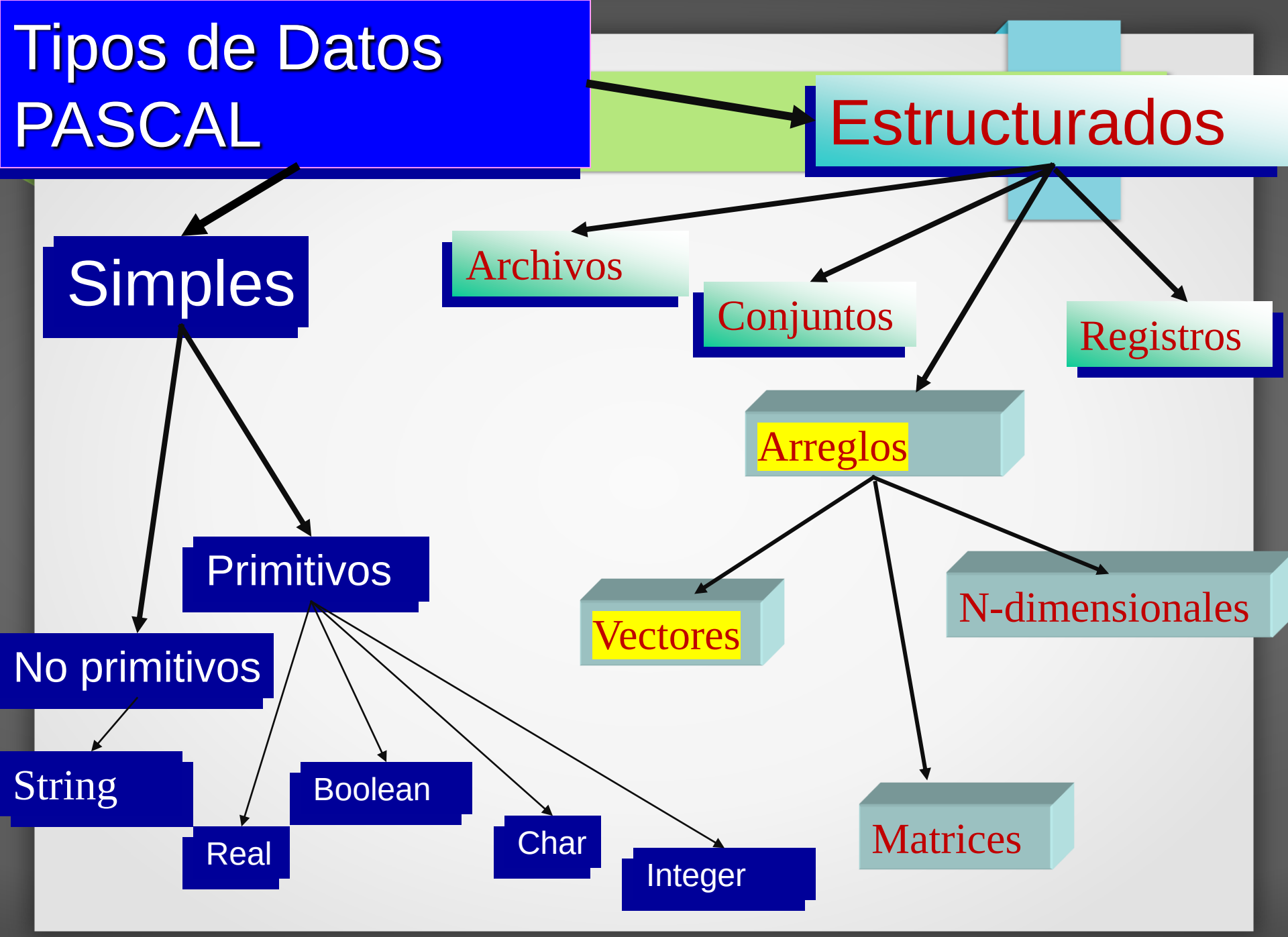
ELABORANDO…

PUDISTE RESOLVERLO?

Seguramente no has podido resolver el problema, porque una vez que has ingresado los datos para procesarlos obteniendo el promedio, ya no los tenés disponibles para mostrar
(en este caso serían las piezas 2, 3 y 5)

Se nos presenta entonces la necesidad de almacenar los datos sobre los que estamos trabajando.
El problema involucra conjuntos de datos asociados entre sí. Si bien estos datos pueden ser almacenados en variables simples, esta solución no facilitaría el procesamiento de la información. Además, no sería una solución de carácter general (la cantidad de información es variable).
Una buena solución es almacenar la información en **variables estructuradas.**

Tipos de Datos PASCAL



Tipos Estructurados

- Es un conjunto de datos reunido bajo un **único** nombre colectivo (su identificador).

- Un tipo de dato estructurado se caracteriza por el(los) tipo(s) de sus componentes y, sobre todo, por el *método de estructuración.*
Cada uno de estos métodos es una opción que puede manejar el programador a través de *la sección TYPE*.

- Un tipo estructurado tiene **más** de una componente, a diferencia de un tipo simple que tiene una sola.

- El usuario tiene que analizar la combinación de datos simples para formar estructuras compuestas que representen apropiadamente la información a procesar; además, debe estudiar como extraer y colocar componentes específicos en dicha estructura.

Datos Estructurados

- Las componentes de un tipo estructurado pueden ser simples o estructuradas.
- En última instancia los datos estructurados deben ser construidos a partir de datos primitivos.
- En el nivel más bajo, las componentes de una variable estructurada son de tipos simples (por lo tanto, cada una de ellas puede ser usada como variables simples: asignadas, leídas, etc.)
- Ejemplo: el número complejo toma la forma de un par de números reales.

Los tipos estructurados del lenguaje PASCAL son:

- Conjuntos
- Arreglos (ARRAY)
- Registros (RECORD)
- Archivos (FILE - TEXT)

En la sección **TYPE** al declarar un tipo de datos estructurado, se describen:

- su *método de estructuración*
- el(los) *tipo*(s) *de sus componentes*

Método de estructuración de los ARREGLOS

- Un **arreglo** es una colección ordenada de variables del **mismo tipo**
- Cada uno de los elementos de la colección se llama **componente del arreglo.**
- Los arreglos poseen un **número fijo** de componentes.
- El acceso a las componentes mediante el **índice del arreglo** (o **subíndice**).

Consideraciones

1.-Tipo homogéneo de componentes

2.-Dimensión física vs lógica

3.- Estructura de acceso aleatorio / directo

Declaración del tipo arreglo lineal (unidimensional)

En la sección **Type** se elige un **identificador** y describe el tipo estableciendo:

a) La **cantidad máxima** de **elementos** que determina el rango de sus **posiciones** (para el acceso individual).

b) El **tipo** de los **valores** que almacenan sus componentes

Identificador de **tipo**

índice

Type

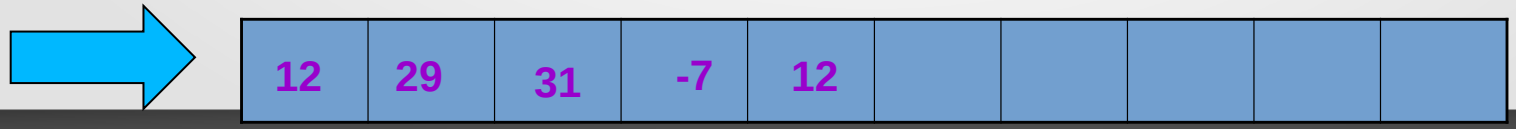
TipoVector = array [**1..10**] of **integer;**

Tipo **base**

El **índice** debe ser **ordinal**.

El **tipo base no** tiene **restricciones** (simple o estructurado)

La declaración del **TipoVector** asocia este identificador a una estructura de **10 elementos** de tipo **entero (base).** Cada uno de ellos se puede acceder mediante su ubicación : **1..10 (índice)**



Declaración de Variables

Var

V1, V2 : TipoVector ;

V1										
	1	2	3	4	5	6	7	8	9	10
V2										

la declaración de una variable tipo arreglo provoca la reserva de espacio en memoria para todas las componentes.

El programador puede decidir qué cantidad de componentes usar (en este caso, hasta 10).

Tendremos por un lado la cantidad máxima posible de elementos: **DIMENSION FISICA** y por otro la cantidad real de elementos en el arreglo: **DIMENSION LOGICA**

Declaración de tipos y acceso a las componentes

```
CONST
   MaxDim=30;
TYPE
   Estacion=1..4;{1:primavera;2:verano;3:otoño;4:invierno}
   TipoVector=array[1..MaxDim] of real;
   Periodo=array[Estacion] of string[15];
   Vec=array['A'..'Z'] of byte;
VAR
  V: Array [1.10] of Integer; {Tmb podría definirse sin uso de Type}
  P:Periodo;
  W:Vec;
  I:Byte;
  C:char;
BEGIN  {Detectar errores, o indicar qué efecto producen las
asignaciones}
   V[2]:= 21.3;
   P[3]:='21/03 - 20/07';
   W['B']:= 0;
   W[‘a']:=45;
   I:=5;    V[I*3]:=V[I]/V[I-3];
   C:= 'Q’; W[C]:= 270;
End.
```

Vectores: lectura y escritura

Las variables de tipo arreglo se acceden componente a componente

Lectura

```
For j:= 1 to 4 do
    Readln(V[j]);
```

Se implementarán mediante procedimientos

Escritura de las componentes de índice par

```
j:= 2;
While j <= 10 do
    begin
      write(V[j]:4);
      j:=j+2;
    end;
```

Vectores: utilidad

Como ya hemos dicho, los arreglos permiten **mantener** en **memoria** un conjunto de datos y volver a acceder a ellos sin necesidad de leerlos nuevamente o regenerarlos.

Veamos a continuación como trabajar con Procedimientos y Funciones que tienen arreglos como parámetros

Leer y escribir un vector de N componentes reales

```
Program LeeEscribe;
type
  TV= array [1..50] of real;
var
   N:Byte;
   Vec:TV;

Procedure LeeVector(Var V: TV; Var N:byte);
Var
  i:byte;
begin
  writeln('Cantidad de números reales a leer <=50');
  readln(N);
  for i:=1 to N do
      begin
        writeln('ingrese el elemento', i);
        readln(V[i]);
      end;
end;
```

```
Procedure EscVector(V: TV; N:byte);
Var
  i:byte;
begin
 for i:=1 to N do
     write(V[i]:5);
end;

Begin
   LeeVector(Vec, N);
   EscVector(Vec, N);
end.
```

Ejercicio

Sea A un arreglo unidimensional de elementos enteros que se ingresa desde un archivo. Desarrollar en un único programa Pascal que permita mediante el uso de un menú, seleccionar una de las opciones siguientes:

a.- Calcular y mostrar la suma de todos los elementos de A
b.- Construir un arreglo B que sea A permutado
c.- Construir un arreglo C con los valores impares de A
d.- Asignar el valor cero a todos los elementos de A de índice impar.
Imprimir los vectores A (original y modificado), B y C cuando corresponda.

Tipos, Variables del Pgm Ppal y Menú

```
PROGRAM E2;
 TYPE
        TVector=array[1..100] of integer;

 VAR {variables del programa principal}
         A,B,C:TVector;
         N,M:integer;
         Opcion:char;

Procedure Menu(Var Op:char);
Begin
 Writeln('Elija una de las opciones');
 Writeln('a - Suma de  todos los elementos');
 Writeln('b - Permutar los elementos');
 Writeln('c – Valores impares');
 Writeln'd-Asignar cero a las componentes de indice impar');
 Writeln('f - finalizar');
 Readln(Op);
End;
```

Ingreso de datos desde archivo

```
Procedure LeeVector(Var A: TVector; Var N:byte);
Var
  Arch:text;
begin
  Assign(Arch, 'Numeros.txt');
  Reset(Arch);
  N:=0;
  While not EOF(Arch) do
    begin
     N:=N + 1;
     read(Arch, A[N]);
    end;
  Close(Arch);
end;
```

Impresión en pantalla e Inciso a)

```
Procedure EscVector(V: TVector; N:byte);
Var
  i:byte;
begin
 for i:=1 to N do
     write(V[i]:5);
end;
Function Suma (A: Tvector; N:byte): longint;
Var
  i:byte;
  S: longint;
begin
  S:=0;
  For i:= 1 to N do
     S:= S + A[i];
  Suma:= S;
End;
```

Inciso c)

```
Procedure Impares(A: Tvector; N:byte; Var C: Tvector;
Var M: byte)
Var
 i: byte;
Begin
M:=0;
for i:=1 to N  do
  if A[i] mod 2 <> 0 then
    begin
     M:=M+1;
     C[M]:=A[i];
    End;
End;
```

Incisos b) y d)

```
Procedure Permuta(A: Tvector; N:byte; Var B: TVector)
Var
 i: byte;
Begin
for i:=1 to N  do
   B[i]:=A[N-i+1];
End;

Procedure AsignaCero(Var A: Tvector; N:byte)
Var
 i: byte;
Begin
 i:=1;
 WHILE i<=N do
  Begin
   A[i]:=0;
   i:=i+2
  end
end;
```

Programa Principal

```
BEGIN {programa principal}
LeeVector(A,N);
EscVector(A,N);
Repeat
    Menu(Opcion);
    CASE Opcion OF
       'a': Writeln('La suma es  ', Suma(A,N));
       'b': begin
              Permutar(A, N, B); EscVector(B,N);
            end;
       'c': begin
              Impares(A,N,C,M);  EscVector(C,M);
            end;
       'd': begin
              AsignaCero(A,N);   EscVector(A,N);
            end;
     END; {case}
until Opcion = 'f'
END.
```